

LABORATORY RECORD**EXP NO.: 10 TROUBLESHOOTING FABRIC APPLICATIONS****AIM**

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

PROCEDURE**Step 1: Navigate to the Fabric Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify Docker Containers

```
docker ps
```

Step 4: Check Peer Logs

```
docker logs peer0.org1.example.com
```

Step 5: Check Orderer Logs

```
docker logs orderer.example.com
```

Step 6: Verify Channel Membership

```
peer channel list
```

Step 7: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 8: Troubleshoot Chaincode Deployment

```
./network.sh deployCC -ccl javascript -ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 9: Verify Chaincode Execution

```
peer chaincode query -C mychannel -n basic \  
-c '{"Args":["GetAllAssets"]}'
```

Step 10: Check Docker Resource Usage

```
docker stats --no-stream
```

Step 11: Restart the Fabric Network

```
./network.sh down  
./network.sh up createChannel -ca
```

Step 12: Clean Docker Resources

```
docker system prune -f
```

PROGRAM

Fabric Network Diagnostics & Troubleshooting Shell Script (troubleshoot_fabric.sh):

```
#!/bin/bash  
# Script for Fabric Application Diagnostics, Error Detection, and Remediation  
  
cd ~/fabric-dev/fabric-samples/test-network  
  
# 1. Start Fabric Network  
./network.sh up createChannel -ca  
  
# 2. Check Container Health and Peer/Orderer Logs  
docker ps  
docker logs peer0.org1.example.com | tail -n 15  
docker logs orderer.example.com | tail -n 15  
  
# 3. Verify Channel Membership & Troubleshoot Chaincode Deployment  
peer channel list  
peer lifecycle chaincode queryinstalled  
  
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/  
chaincode-javascript  
  
# 4. Query Ledger & Monitor System Resource Allocation  
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'  
docker stats --no-stream  
  
# 5. Network Teardown, Environment Reset & Resource Pruning  
./network.sh down  
./network.sh up createChannel -ca  
./network.sh down  
docker system prune -f  
  
echo "Troubleshooting Diagnostic Workflow Executed Successfully."
```

OUTPUT

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -
ca
Creating channel 'mychannel'... done
Starting peer0.org1...
Starting peer0.org2...
Starting orderer.example.com...
Fabric Network Started Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker ps
CONTAINER ID   IMAGE                                COMMAND
CREATED       STATUS      NAMES
8a9f21b10c9d   hyperledger/fabric-peer:2.5.0      "peer node start"      2 minutes
ago          Up 2 minutes peer0.org1.example.com
3c2b11e9a4f2   hyperledger/fabric-peer:2.5.0      "peer node start"      2 minutes
ago          Up 2 minutes peer0.org2.example.com
1f9a88e22d3b   hyperledger/fabric-orderer:2.5.0    "orderer"              2 minutes
ago          Up 2 minutes orderer.example.com
d4e5f6a1b2c3   hyperledger/fabric-ca:1.5.6         "sh -c 'fabric-ca-server" 2 minutes
ago          Up 2 minutes ca_org1
a1b2c3d4e5f6   hyperledger/fabric-ca:1.5.6         "sh -c 'fabric-ca-server" 2 minutes
ago          Up 2 minutes ca_org2

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker logs
peer0.org1.example.com | tail -n 3
Peer Started Successfully
Listening on Port 7051
Joined Channel Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-
network$ docker logs orderer.example.com | tail -n 3
Orderer Started Successfully
Receiving Transactions
Creating New Blocks
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer channel list
Channels peers has joined:
mychannel

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode
queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:a7629b3a783e49321c8410d8a5f829a3, Label: basic_1.0

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl
javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Packaging Chaincode...
Installing Chaincode...
Approving Chaincode...
Committing Chaincode...
Chaincode deployed successfully
```

```

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[
  {"ID": "asset1", "Color": "blue", "Size": 5, "Owner": "Tomoko", "AppraisedValue":
300},
  {"ID": "asset2", "Color": "red", "Size": 7, "Owner": "Brad", "AppraisedValue": 400}
]

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker stats --no-stream
CONTAINER ID   NAME                                CPU %       MEM USAGE / LIMIT   MEM %       NET
I/O
8a9f21b10c9d   peer0.org1.example.com             3.12%       68.4MiB / 7.76GiB   0.86%
1.2MB / 850KB
3c2b11e9a4f2   peer0.org2.example.com             2.85%       65.1MiB / 7.76GiB   0.82%
1.1MB / 820KB
1f9a88e22d3b   orderer.example.com                1.04%       32.8MiB / 7.76GiB   0.41%
890KB / 1.5MB

```

```

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping Containers...
Removing Network...

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -
ca
Network Restarted Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ docker system prune -f
Deleted Containers: 8b2c11e34f01
Deleted Networks: fabric_test
Deleted Images: unused_image_id
Total reclaimed space: 450.2MB

```

RESULT

The Experiment was done successfully.